

Kaixin Chen¹

¹Zhejiang University Hangzhou

February 20, 2026

HwOS: A Thread-Level RTL Abstraction for Composable and Observable Hardware Design

Kaixin Chen
Zhejiang University
Hangzhou, China
Email: 3230100656@zju.edu.cn

Abstract—With the rise of heterogeneous computing, hardware control logic complexity has grown exponentially, causing state space explosion in traditional RTL design. This paper proposes HwOS, a hardware construction framework that introduces OS-level abstractions into the RTL domain. HwOS defines Thread-Level RTL (TL-RTL), encapsulating sequential logic into HardwareThread objects with independent lifecycles. By replacing fragmented FSMs and low-level handshakes with a Fork-Join concurrency model and a “Service-Based Pipeline” paradigm, HwOS decouples centralized control into a distributed, temporally continuous thread context. To address verification challenges, HwOS provides HwOSgdb, enabling semantic-level observability via DPI-based state monitoring. Evaluation demonstrates that HwOS significantly reduces control-logic code volume with minimal area overhead, effectively accelerating agile hardware development.

Index Terms—Hardware Description Language, Thread-Level RTL, Service-Based Pipeline, Design Automation, Hardware Debugging.

I. INTRODUCTION

A. Background and Challenges

To break through the power wall and pursue extreme performance, modern computer architecture design is gradually evolving towards heterogeneity and specialization. [1] Whether it is the out-of-order execution scheduling of processors or the complex data flow control of accelerators, the core pain point points to the increasingly bloated Control Logic. In traditional hardware description languages (such as Verilog/SystemVerilog [2]), designers are forced to view the system from a microscopic “signal-level” perspective, manually managing FSM state transitions, pipeline register backpressure [3], and complex inter-module handshake protocols. This low-level abstraction causes design intent to be submerged in tedious signal wiring, vastly increasing verification costs and limiting module reusability.

B. Limitations of Existing Approaches

The use of high-level programming techniques in hardware design can be traced back to early functional HDLs like Lava [4], which leveraged Haskell’s features for circuit specification and verification. To improve design productivity, modern hardware construction languages represented by Chisel [5] and Bluespec [6] have introduced mechanisms such

as metaprogramming, parameterized generation, and Atomic Rules. These tools often rely on advanced compiler frameworks like FIRRTL [7] to perform hardware transformations. While these tools have successfully improved the efficiency of building datapaths, a semantic gap remains when describing long-latency operations and dynamic control flow.

High-Level Synthesis (HLS) methodologies, e.g. PyMTL [8] LegUp [9], attempt to raise the abstraction level to higher-order language to bridge the hardware-software gap. Although HLS tools significantly improve algorithmic productivity [10], they often struggle with fine-grained timing control and structural transparency. This leads to unpredictable performance when implementing complex control-centric logic, such as out-of-order execution [11]. In existing frameworks, designers still need to manually map sequential behaviors into fragmented FSM slices across different modules. There is a lack of a native abstraction that can intuitively describe the “lifecycle” of hardware operations while maintaining the structural precision of RTL. Furthermore, while earlier projects such as ReconOS and BORPH [12] [13] pioneered the integration of OS services into reconfigurable hardware through software-hardware co-design or custom kernels, they primarily focused on the system level. HwOS, by contrast, embeds these abstractions directly within the RTL domain to address the micro-architectural control complexity.

C. Core Philosophy of HwOS

Inspired by operating system process management and microprogramming design [14] in CISC, this paper proposes **HwOS**. Its core philosophy is: “**Hardware is an Operating System, Everything is a Thread.**” HwOS does not aim to run a software OS on hardware, but rather uses metaprogramming techniques to statically map the concepts of Thread, Process, Driver, and Resource from an OS into efficient RTL circuits. We propose a “**Thread-Level RTL**” (TL-RTL) abstraction, allowing designers to describe hardware timing behavior in an imperative style, while delegating tedious FSM generation and resource arbitration to the compiler.

D. Contributions

The main contributions of this paper are summarized as follows:

The source code of HwOS is available at <https://github.com/NullStarfish/HwOS>.

1) *Proposal of a Thread-Level RTL Primitive System:* We designed a temporally complete set of primitives ('Entry', 'Step', 'Wait', 'Fork'), supporting thread-to-thread communication with zero wiring overhead via closures, while ensuring constructive deadlock freedom and Abort safety.

2) *Construction of a Driver-Based Control Flow-Resource Model:* We redefined the role of the Driver, making it a **Control Flow Proxy**. Through an "Instruction Injection" mechanism, the Driver can automatically translate resource contention into thread stall logic, implementing decentralized resource arbitration.

3) *Proposal of the "Service-Based Pipeline" Paradigm:* Based on HwOS abstractions, this paper proposes a new pattern for building computational units. This pattern encapsulates functional units as "Services" and models instructions as active "Threads." Experiments show that this "**Instruction-as-Thread**" model can naturally implement out-of-order issue and dynamic dependency resolution without a centralized hazard detection unit.

4) *Implementation of Fractal Architecture and Native Observability:* Using a recursive Kernel-Process-Thread structure, we achieved modular reuse of hardware logic and developed the HwOSgdb toolchain. Through the DPI interface and automatically generated symbol tables, we realized a semantic mapping between RTL simulation states and source-level thread logic.

II. CORE ARCHITECTURE OF THREAD-LEVEL RTL

The cornerstone of HwOS is the elevation of hardware sequential logic from flat FSMs to a hierarchical thread model. Inspired by Complex Instruction Set Computer (CISC) microcode [14] architectures and concurrency models [15] in high-level programming languages, we propose a set of metaprogramming primitives based on `HardwareThread`. Leveraging Chisel's metaprogramming features [5] and Scala's closure mechanism, HwOS can automatically compile imperative thread descriptions into efficient low-level RTL logic during the elaboration phase.

A. Formal Definition

A hardware thread T is formally defined as a tuple $M = (S, \Sigma, \delta, s_0, F, \Omega)$, where:

State Space (S):

$S = \{PC \in \mathbb{N}, Active \in \{0, 1\}\}$, representing the virtual program counter and execution status.

Input Alphabet (Σ):

Input predicates derived from hardware signals (e.g., *start*, *abort*) and user conditions (*cond*).

Transition Function (δ):

A mapping $\delta : S \times \Sigma \rightarrow S$. For a blocking step with predicate C :

$$PC_{t+1} = \begin{cases} PC_t & \text{if } \neg C \\ PC_t + 1 & \text{if } C \end{cases} \quad (1)$$

Initial State (s_0):

$s_0 = (PC = 0, Active = 0)$, indicating a reset to idle at address 0.

Final State (F):

Transition $Active \rightarrow 0$ triggered by the `exit()` primitive.

Output Function (Ω):

$\Omega : S \times \Sigma \rightarrow \text{Signals}$ that drives managed wires or registered outputs.

B. Primitive Design

1) *Entry, Step, and Implicit Control:* HwOS abandons explicit state enumeration in favor of an implicit Program Counter (PC) to manage control flow.

- **Entry (Entry Scope):** `entry` defines the hardware generation boundary. All sequential logic must be declared within this scope to provide context capture.
- **Step (Temporal Slice):** The basic scheduling unit. Each Step corresponds to FSM states, with PCs assigned automatically to construct the transition graph.
- **PC (Virtual Program Counter):** Similar to `μpc` in microcode, `pc` is a state register maintained by the kernel to linearize discrete logic into sequential semantics.

```
t.Step {
  // Unconditional jump to implement a loop
  t.pc := 0.U
}
```

Listing 1. Recursive loop implementation in HwOS

2) *Blocking Primitives and Guarded Execution:* To express common handshake and backpressure, HwOS provides predicate-based blocking primitives mapped to self-loop logic:

- **waitCondition(cond):** Holds the current state while the condition is not met. Implemented as `when(!cond){pc:=pc}`.
- **waitAndAct(cond){action}: A blocking primitive with side effects.** The FSM advances and executes `action` only when the condition is met.

3) Lifecycle and Concurrent Monitoring:

`HardwareThread` maintains a complete FSM lifecycle with independent states:

- **Control Interface:** `start()` triggers launch, `exit()` terminates execution, and `abort()` forces a reset. Static checks ensure all paths eventually exit.
- **State Observability:** `isRunning`, `done`, and `pc` are exposed as module interfaces for real-time monitoring by debuggers.
- **Global Scope:** Logic defined in `Global` executes in parallel with the FSM, useful for interrupts or concurrent monitoring independent of the PC.

4) *Hierarchical Multi-Threading via Closures:* HwOS leverages Scala's Lexical Scoping to implement zero-wiring-overhead thread communication.

- **fork/prepareThread:** Used to instantiate and launch child threads. `fork` provides a shortcut for launching within a specific step.

- **Implicit Context Capture:** Child threads defined within a parent's entry closure can directly access the parent's registers and wires without manual port punching.

```
cpu.entry {
  val accReg = RegInit(0.U(32.W))
  val mainReg = RegInit(0.U(32.W))
  cpu.fork("Accelerator") {
    val t = ContextScope.current match {
      case ThreadCtx(t) => t
      case _ => null
    }
    t.Step("Load") { accReg := 10.U }
    t.Step("Compute") { accReg := accReg * 2.U }
    t.Step("Store") { accReg := accReg + 5.U }
    t.Step("Done") { t.exit() }
  } { // callback closure
    cpu.pc := 5.U
    mainReg := accReg
  }
}
```

Listing 2. Hierarchical thread communication via closures

5) *Configurable Mealy/Moore Machines:* The compiler supports parameterizing threads as Moore or Mealy machines. Mealy threads allow the first `Step` logic to take effect in the cycle `start()` is asserted, while Moore threads wait one cycle. This allows performance-area trade-offs without rewriting logic.

6) *Context-Aware Type System:* `ContextScope` uses a compile-time stack to track execution context and prevent illegal descriptions:

- **LogicCtx:** Pure combinational context for stateless `HardwareLogic`.
- **ThreadCtx:** Default sequential context within `entry`, allowing register definitions and `Step` declarations.
- **AtomicCtx:** Context inside a `Step` block, where nesting new `Steps` is strictly prohibited.

7) *Correct-by-Construction Abort Safety:* HwOS ensures system integrity during exceptions, a challenge often addressed in the theory of latency-insensitive design [3] by decoupling computation from communication:

- **Forced Reset:** `abort()` resets the PC to zero and deactivates the thread with the highest priority.
- **Managed Outputs:** Wrapping external signals with `driveManaged` generates Muxes that force safe "Idle" values during an Abort, ensuring datapath safety.

8) *Temporal Completeness and Compilation:* HwOS primitives cover all standard FSM behaviors, as summarized in Table I. The compiler utilizes a two-phase strategy: a **Collection Phase** to capture logic as Thunks, followed by a **Generation Phase** to expand the switch-case structure and optimize the PC allocation.

III. FRACTAL ARCHITECTURE AND KERNEL MANAGEMENT

To address modularity and reuse challenges in large-scale hardware design, HwOS abandons the flat and static module instantiation method of Verilog in favor of an OS-like **Fractal Architecture**. This architecture consists of three levels:

TABLE I
MAPPING HWOS PRIMITIVES TO STANDARD FSM BEHAVIORS

FSM Behavior	HwOS Primitive	Timing Characteristic
Transition	Step { ... }	Logic executes; next PC next cycle.
Wait State	waitCondition(cond)	State holds until condition met.
Guarded Act	waitAndAct(c){a}	Action suppressed if <i>c</i> unmet.
Mealy Output	driveManaged	Combinational output via Mux.
Sync Reset	abort()	Immediate reset to PC=0.

Kernel, `HwProcess`, and `HardwareThread`, supporting seamless scaling from single functional units to many-core systems via recursive composition. [16]

A. Recursive Logic Container: *HwProcess*

`HwProcess` serves as the logic container and namespace boundary in HwOS, analogous to a process in an operating system or a directory in a file system. It encapsulates specific hardware logic while maintaining the system's hierarchical relationship.

- **Recursive Composition:** Each `HwProcess` instance holds a reference to its Parent and maintains a list of Children processes. Through the `spawn` interface, a process can dynamically instantiate any number of child processes. This recursive structure allows designers to build hardware in a "self-similar" manner. For example, a "Multi-Core GPU" process can spawn multiple "Compute Core" processes, and a "Compute Core" process can in turn spawn "ALU" and "LSU" processes.
- **Hierarchical Namespace:** To precisely locate logic units during debugging, HwOS adopts a path-based naming rule. The full name of a child node is formed by concatenating the parent node's name prefix (e.g., GPU/Core0/FetchUnit). This naming mechanism naturally eliminates naming conflicts and provides a semantic index for automated debugging tools.

B. Leaf Execution Unit: *HardwareThread*

While the Process is a static container, the `HardwareThread` is the dynamic execution entity residing at the leaf nodes of the fractal structure.

- **Minimal Scheduling Entity:** The Thread is the only object in HwOS that possesses a Program Counter (PC) and state machine logic. It attaches to a specific Process, but its scheduling and execution are independent.
- **Factory Pattern Generation:** Threads must be created via the Process's `createThread` method. This factory pattern ensures that every thread is automatically registered in the management list of its owning Process at creation time, establishing a strict Owner-Member relationship.

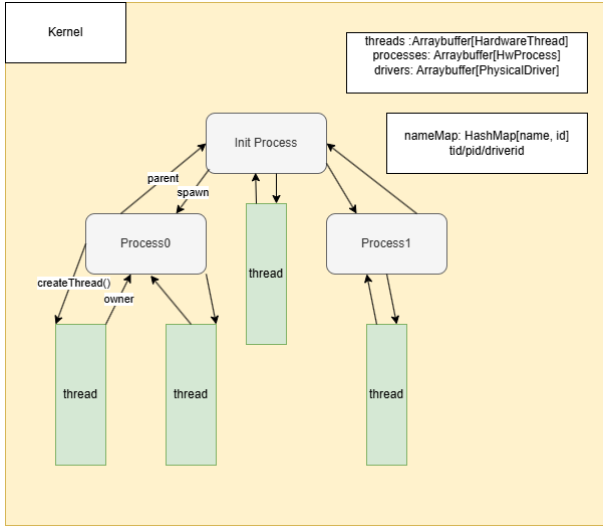


Fig. 1. Fractal Architecture: The recursive composition of Processes and Threads managed by the global Kernel, featuring flattened ID allocation for simplified hardware mapping.

C. Global Resource Registry: Kernel

The Kernel is the core Singleton (at elaboration time) of HwOS, acting as the global manager with a "God's Eye View." Although the logic view of HwOS is tree-like, to adapt to the flat nature of hardware circuits, the Kernel performs **Hierarchy Flattening** during the compilation phase.

- **Global ID Allocation:** The Kernel maintains a global registry of all Threads and Processes. During the Elaboration phase, the Kernel traverses the entire process tree, assigning a unique TID (Thread ID) to each registered `HardwareThread` and a PID (Process ID) to each `HwProcess`.
- **Driver Mounting:** The Kernel provides a mount interface for mounting hardware drivers (`PhysicalDriver`). Once a driver is mounted, it acquires a globally unique `DriverID` and becomes visible to all threads within the Kernel's unified namespace. This mechanism breaks the restriction in Verilog where interfaces must be passed through layers via "Port Punching," realizing a service discovery mechanism similar to OS system calls.

D. Unification of Static Build and Dynamic Behavior

The fractal architecture of HwOS ingeniously unifies Scala's runtime behavior with the static structure of hardware.

- **Construction Lifecycle:** When the top-level Module is instantiated, the Kernel starts the build process. The `build()` method of `HwProcess` is called recursively, triggering logic execution in `entry()`. At this point, `createThread` and `spawn` in the Scala code are executed, completing the static expansion of the entire hardware tree.
- **Monitoring and Debugging Support:** Based on the flattened TID mapping, the Kernel can generate a system-

wide Symbol Table and aggregate the runtime state of all threads (e.g., `pc`, `active`, `done`) onto a unified monitoring bus via the DPI interface. This allows external debuggers (like `HwOSgdb`) to access the state of deeply nested submodules with $O(1)$ complexity without caring about their depth in the hierarchy.

IV. THE CONTROL-FLOW AND RESOURCE MODEL: INTRODUCING DRIVERS

In operating system semantics, besides threads as execution entities, mutual exclusion and synchronization of core resources are crucial. HwOS introduces the **Driver** model in the TL-RTL abstraction, defining it as middleware connecting Control Flow (Thread) and Physical Resources (Resource). The Driver not only encapsulates timing details of physical interfaces but, more critically, acts as a "Control Flow Proxy," taking over access permission management of resources from threads

A. Driver as the Unique Entry for Resource Access

In HwOS design specifications, physical resources (such as Register Files, SRAM, AXI bus controllers, etc.) are strictly encapsulated within Drivers. HwOS enforces the **"Access via Proxy"** principle: all resource interactions must occur through APIs provided by the Driver; threads are strictly prohibited from directly reading or writing underlying signals. This design ensures atomicity and consistency of resource access, making underlying hardware implementation details (such as read/write port counts and timing parameters) transparent to upper-layer logic, thereby achieving "Information Hiding" in hardware implementation. This architectural choice is effectively a hardware implementation of the monitor concept [17], a classic OS structuring principle. By encapsulating physical resources within a Driver, HwOS ensures that all interactions are governed by implicit synchronization and mutual exclusion logic.

B. Timing Specification of Function APIs

APIs provided by Drivers possess **Context-Aware** characteristics Utilizing the `ContextScope` mechanism, Drivers can dynamically generate different hardware structures based on the caller's context:

- **Thread Context (ThreadCtx):** When an API is called within an `entry` block, the Driver automatically injects sequential logic (e.g., state machine nodes) and utilizes registers to latch return values, ensuring cross-cycle signal stability
- **Combinational Context (AtomicCtx/LogicCtx):** When an API is called within a combinational logic block, the Driver generates only instantaneous combinational logic paths or returns current cycle wire values, introducing no additional state overhead

This polymorphism allows the same set of APIs to be used for building complex sequential flows as well as fast combinational logic calculations, greatly improving code reuse

C. Driver as a Control Flow Proxy

Drivers in HwOS discard the traditional "Request-Response" handshake model in favor of an "**Instruction Injection**" mechanism. The Driver itself does not maintain an independent control flow state machine but acts as a proxy, directly intervening in the calling thread's control flow.

Specifically, when a thread calls `driver.read(addr)`, the Driver "injects" a new Step into the current thread's execution sequence. In this Step, the Driver automatically generates arbitration request logic (e.g., `checkConflict`) and inserts a `waitCondition(!stall)` statement. This means pipeline Stall logic caused by resource contention is automatically woven into the user thread by the Driver, allowing users to implement safe resource access without explicitly writing handshake code.

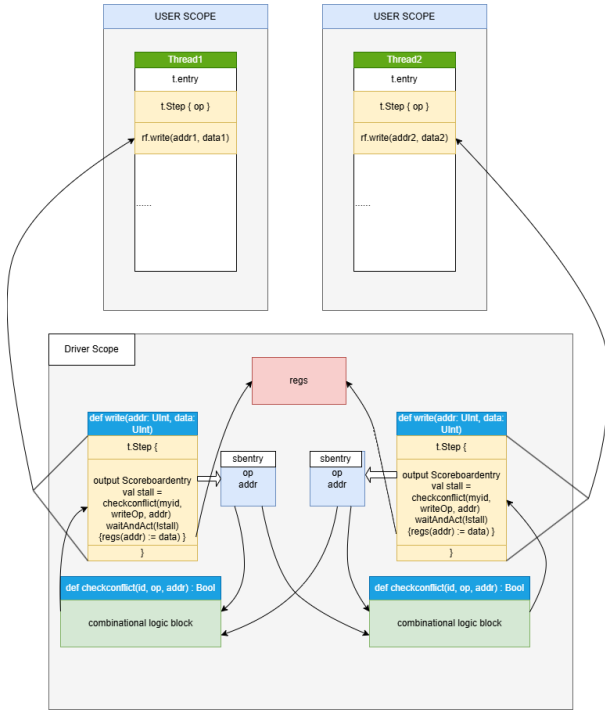


Fig. 2. The Driver Architecture: acting as a bridge between User Scope threads and physical resources, injecting stall logic and handling arbitration.

D. Control Flow Inversion via Callbacks

To implement efficient atomic operations at the hardware level, Driver APIs widely adopt a higher-order function-based Callback mechanism, realizing **Inversion of Control (IoC)**.

Taking `readAtomic(addr) (callback)` as an example, the logic in the callback closure is not executed immediately but is encapsulated as a block of combinational logic, mounted onto the clock cycle where resource arbitration is granted (Grant). When the arbiter determines the resource is available, the Driver activates the callback logic and completes data transfer within the same cycle. This mechanism eliminates the extra cycle overhead common in traditional "Request-Ack-Read" handshake protocols, achieving true Zero-Latency back-to-back access.

E. Driver as a Peer Communication Hub

In HwOS, the Driver also acts as a hub for implicit inter-thread communication. For example, in `ScoreboardRegfileDriver`, through the `ScoreboardEntry` structure, the Driver maintains a globally visible **Intent Bus**.

All threads attempting to access resources publish their operation intents (e.g., Read/Write, Address) to this bus. The combinational logic inside the Driver scans all intents in real-time, calculating the stall signal for each thread based on preset priority policies. This mechanism allows threads to perceive each other's existence and automatically coordinate execution order by simply sharing a Driver, without explicit signal wiring, essentially implementing a distributed synchronization mechanism.

F. DriverMeta and Automatic Configuration

To adapt to different performance and area requirements, HwOS introduces the `DriverMeta` metadata description system, supporting highly parameterized resource modeling.

Developers can flexibly adjust the Driver's multi-port arbitration logic by configuring parameters like `read_clients` and `write_clients`, or even inject custom conflict detection algorithms (such as `RW_Lock` or `Full_Mutex`) via the `conflict_policy` function. This allows the same Driver template to be instantiated as a high-performance multi-port Register File or a low-area single-port RAM controller without modifying upper-layer thread logic.

G. Flexibility of Scheduling Logic

Thanks to the "Instruction Injection" model, HwOS achieves complete decoupling of scheduling policies from functional logic. Whether using simple blocking access, high-performance Pipelined access, or out-of-order access supporting Forwarding, the difference lies only in the structure of the Step injected inside the Driver.

Experiments show that transforming a sequentially executed CPU into a pipeline architecture supporting out-of-order issue requires only replacing the underlying Driver implementation (e.g., switching from `ScoreboardRegfileDriver` to `PipelinedScoreboardDriver`), with almost no changes to the upper-layer instruction issue logic. Such flexibility is unimaginable in traditional RTL design.

V. SERVICE-BASED PIPELINE ARCHITECTURE: STITCHING FSMs FOR SERVICES

Traditional processor pipeline design is essentially an engineering practice of "Spatial Partitioning." To improve throughput, the execution process of an instruction is rigidly cut into multiple independent combinational logic blocks (e.g., Fetch, Decode, Execute). These blocks are spatially separated and connected via pipeline registers and complex handshake signals (e.g., Valid-Ready protocols). While effective, this design leads to serious "Context Fragmentation": to maintain instruction identity, control signals and data must be explicitly

passed between pipeline stages, resulting in complex cross-module wiring and state synchronization overhead.

HwOS proposes a **Service-Based Pipeline** architecture based on "Temporal Stitching." Through metaprogramming techniques, we re-"stitch" the Finite State Machine (FSM) logic, traditionally scattered across various pipeline stages, into a semantically coherent whole, abstracting it as an independent hardware thread.

A. Core Paradigm Shift: Instruction as a Thread

In the HwOS architecture, we abandon the traditional view of "instructions as passive data flowing through fixed pipes" in favor of a new paradigm: **"instructions as active entities autonomously seeking services."** Every executing instruction is instantiated as an independent `HardwareThread`. The "Instruction-as-Thread" model enables distributed out-of-order execution, conceptually similar to the Tomasulo algorithm used in high-performance processors like the MIPS R10000. [11] [18]

This thread utilizes the HwOS `Step` mechanism to define the series of logical operations it needs to undergo during its lifecycle. Although these operations physically correspond to the activation of different hardware units across different clock cycles, at the designer's source level, they are written as a continuous `Closure`.

This "Instruction Thread" model implements **Temporal Stitching** of control flow:

```
// Linear view of an instruction (Pseudo-code)
entry {
  Step("Fetch") { ... } // Access Fetch Service
  Step("Decode") { ... } // Decode Instruction
  Step("Execute") {
    // Fork child threads to access specific
    // functional unit services
    if (isALU) fork(aluDriver, ...)
    if (isLSU) fork(lsuDriver, ...)
  }
  Step("Writeback") { ... } // Wait for result and
  write back
}
```

Listing 3. Linear view of an instruction thread in HwOS

Although the underlying hardware implementation still requires distributed execution of these steps in the time domain, the Chisel compiler (acting as the "Stitching Machine") automatically analyzes the context dependencies between these Steps, generating a macro state machine that unifies state transitions and variable retention. Designers are freed from the burden of maintaining multiple scattered small FSMs and can focus on describing linear instruction execution flows.

B. Replacing Explicit Handshakes with Thread Forking

To coordinate pipeline stages with different rates, traditional designs rely on explicit inter-module handshake interfaces (e.g., Valid-Ready). Although Valid-Ready supports non-blocking initiation, in complex backpressure scenarios, upstream module state machines often have to include explicit wait states to respond to downstream congestion, resulting in tight timing coupling between modules.

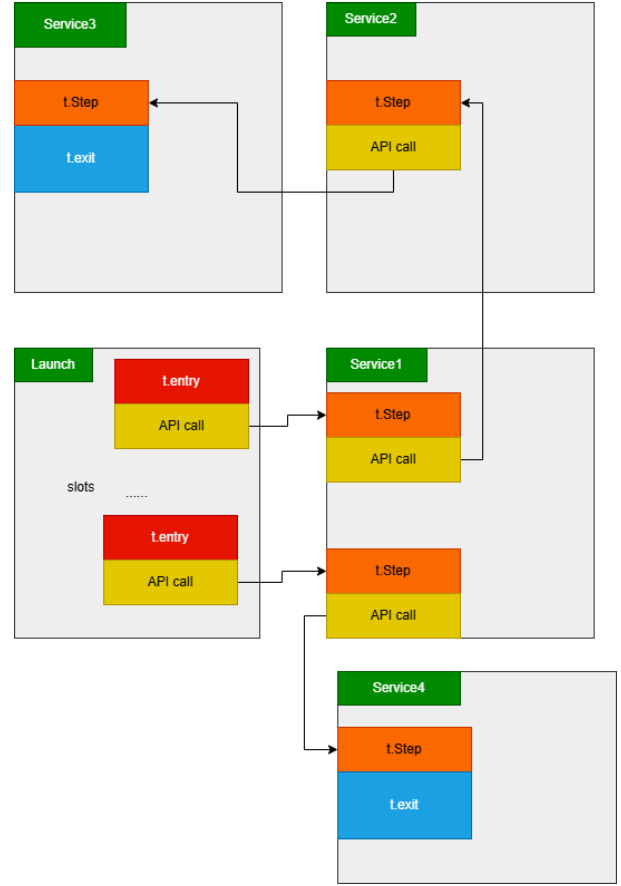


Fig. 3. Service-Based Pipeline: Instructions operate as independent threads, invoking functional units as services via asynchronous calls.

HwOS decouples this interaction via the thread model. When the main thread needs to access a service that may incur latency (e.g., accessing memory or complex ALU operations), it no longer directly initiates a Valid-Ready handshake but uses the `fork` primitive to spawn a transient "Service Child Thread".

- **Control Flow Decoupling:** The main thread executes the `fork` action typically within a single cycle ("Fire-and-Forget"), after which it can immediately enter the next logical state (e.g., checking the dependency scoreboard or processing the next instruction) without waiting for the service to complete.
- **Synchronization Encapsulation:** Physical-level interface synchronization (including potential Valid-Ready backpressure handling) is completely encapsulated within the spawned service child thread. The lifecycle of the child thread is strictly bound to the physical transaction it serves.

This mechanism transforms complex interface timing synchronization problems into thread lifecycle management problems. It replaces rigid Cycle-level Coupling with flexible **Task-level Concurrency**, greatly simplifying the design complexity of core control logic.

C. Service-Oriented Encapsulation of Hardware Units

To support the operation of "Instruction Threads," functional units in the processor (e.g., ALU, LSU, Branch Unit) are no longer bare combinational logic blocks but are encapsulated as standard HwOS Drivers, i.e., "Services".

The key to this service-oriented encapsulation is that it abstracts underlying interaction protocols:

- **Protocol Hiding:** Underlying hardware units may use blocking or non-blocking protocols, credit-based flow control, or other complex handshake mechanisms. Drivers hide these details behind a unified API (e.g., `read`, `write`, or atomic operations).
- **Asynchronous Interaction:** Instruction threads request services by calling Driver interfaces. Thanks to HwOS's thread scheduling mechanisms, these calls can be asynchronous. After issuing a request, a thread can suspend to wait for a result, or spawn a child thread to handle time-consuming operations, freeing the main thread to perform other tasks.

D. Dynamic Scheduling and Dependency Management

In a Service-Based Pipeline, multiple "Instruction Threads" exist concurrently in a thread pool. They dynamically initiate requests to various "Service Drivers" based on their execution progress. This naturally forms a distributed out-of-order execution model without building a massive centralized Issue Queue found in traditional superscalar processors.

To ensure sequential consistency of execution results, we introduce service-based dependency management mechanisms, such as `PipelinedScoreboard`. Acting as a special Driver, it maintains the occupancy state (Busy Bits) of the register file. Before requesting computation services (e.g., ALU operations), an instruction thread must query and reserve target register resources from the Scoreboard service. If resources are occupied, the current instruction thread automatically enters a suspended state, waiting for the preceding thread holding the resource to release it.

VI. EVALUATION

To comprehensively evaluate the HwOS framework in terms of hardware resource consumption, Timing Performance, and code expressiveness, we designed experiments across four dimensions:

- 1) **Basic Micro-benchmark:** Comparing overhead of basic control flow (jumps, loops).
- 2) **Concurrency Micro-benchmark:** Evaluating the hardware cost of thread forking, context capture, and callback mechanisms.
- 3) **Pipeline Macro-benchmark:** Evaluating the comprehensive performance of the Service-Based Pipeline in implementing complex out-of-order control.
- 4) **Many-Core Scheduling Test:** Evaluating resource overhead of HwOS during parameterized expansion.

All experimental code was generated into SystemVerilog by the HwOS compiler and synthesized/analyzed using **Yosys**

(Open Source Synthesis Suite) targeting Xilinx Artix-7 Series FPGAs (`synth_xilinx`). **Estimated LCs** (Logic Cells), **Registers**, and **Data Path Delay** were used as core evaluation metrics.

A. Micro-benchmark: Validation of Zero Timing Penalty

We selected classic control logic containing state transitions, conditional waits (`waitCondition`), and counting loops, implementing both an HwOS version and a functionally equivalent handwritten SystemVerilog (SV) version.

TABLE II
MICRO-BENCHMARK RESOURCE AND TIMING COMPARISON

Metric	HwOS Thread	SV Baseline	Delta
Registers (FDRE)	36	36	0%
Logic Cells (LCs)	45	43	+4.6%
Critical Path Delay	1041 ps	1041 ps	0%
Logic Levels	2 (LUTs)	2 (LUTs)	0%

Analysis:

- **Zero Timing Penalty:** The critical path delay for both versions is exactly **1041 ps**. This proves that the FSM structure generated by HwOS is optimal. The compiler successfully "flattens" high-level abstractions into underlying circuit structures (FDRE → LUT → FDRE) consistent with handwritten code.
- **Minimal Area Overhead:** HwOS consumed only 2 more LUTs. Netlist analysis reveals this stems from slightly more generic PC decoding logic, which is negligible after FPGA mapping.

B. Concurrency Benchmark: Cost of Fork and Closures

To evaluate the hardware cost of **Non-blocking Thread Forking** and **Closure Context Capture**, we designed the `ForkNonBlockingModule` experiment. A parent thread (CPU) dynamically forks a child thread (Accelerator) which accesses parent registers (closure capture) and interrupts the parent via a callback.

TABLE III
FORK AND CALLBACK MECHANISM EVALUATION

Metric	HwOS Fork	SV Baseline	Delta
Registers (FDRE)	72	73	-1.4%
Logic Cells (LCs)	78	75	+4.0%
Critical Path	1452 ps	1150 ps	+26.2%

Analysis:

- **Efficient State Management:** HwOS uses fewer registers (-1.4%) due to the `ContextScope` mechanism, which efficiently infers variable sharing logic and avoids redundant state registers common in manual coding.
- **Timing Trade-off:** The 26% increase in delay is due to the priority logic of the Callback mechanism. HwOS inserts extra multiplexers to ensure `Global` callbacks

have higher priority than normal steps. This is a reasonable trade-off for the development efficiency gained by eliminating manual handshake management.

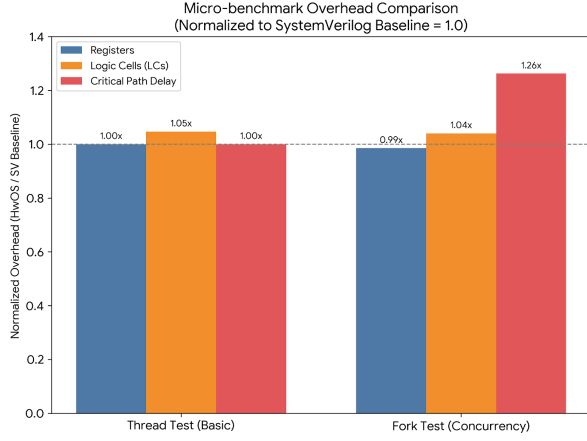


Fig. 4. Normalized Overhead Comparison: HwOS vs. SystemVerilog Baseline across basic and concurrent benchmarks.

C. Macro-benchmark: Service-Based Out-of-Order Pipeline

We synthesized a PipelineModule simulating a heterogeneous superscalar processor core. It includes ALU/LSU Drivers and a Pipelined Scoreboard. We launched 5 concurrent instruction threads simulating a Basic Block with RAW, Structural, and Memory dependencies.

TABLE IV
SERVICE-BASED PIPELINE SYNTHESIS RESULTS

Metric	Value	Analysis
Total Cells	476	Includes all thread logic & resources
Registers	148	State for 5 threads, Scoreboard, RegFile
Est. LCs	107	Core Highlight: Full OoO control
Max Freq	~475 MHz	Critical Path 2.083 ns (Artix-7)
LoC	~150 Lines	>60% reduction vs Verilog

Analysis: The control logic consumes only **107 Logic Cells**, attributed to the "Instruction Injection" mechanism. Dependency detection is distributed into each thread's Step state machine, avoiding the area-heavy CAM logic of centralized Issue Queues. The system achieves a high frequency (~475 MHz) because the decoupled control flow naturally breaks critical paths, keeping logic levels within 4 stages.

D. Scalability Benchmark: Many-Core Process Scheduling

We evaluated the fractal architecture's scalability using a MultiCoreGpuModule with a dynamic GpuDispatcher and N parallel ComputeCore processes ($N \in \{1, 2, 4, 8\}$), all competing for VRAM via a shared driver.

Analysis:

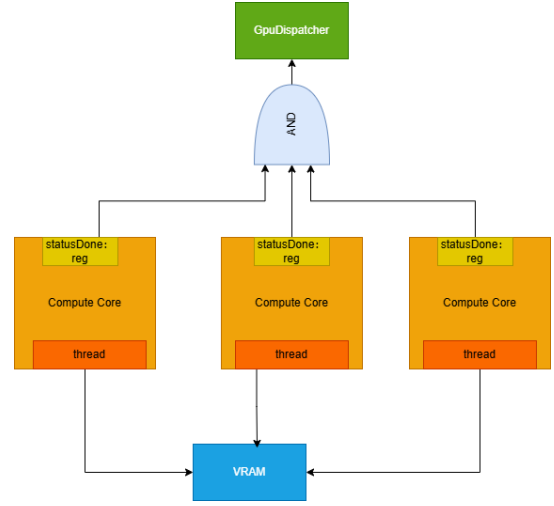


Fig. 5. Many-Core Experiment Architecture: Dispatcher and Compute Cores competing for shared VRAM resources.

TABLE V
MANY-CORE SCHEDULER SYNTHESIS RESULTS (ARTIX-7)

Cores	Regs	LCs	Max Freq	Crit. Path
1	74	54	~788 MHz	1.268 ns
2	143	293	~571 MHz	1.750 ns
4	281	516	~302 MHz	3.302 ns
8	557	2181	~243 MHz	4.103 ns

- 1) **Linear Scalability of State:** Register consumption grows linearly ($O(N)$), introducing 70 registers per core. This confirms the low marginal cost of the HwOS kernel.
- 2) **Adaptive Arbitration:** The jump from 1 to 2 cores (54 to 293 LCs) illustrates HwOS's zero-overhead abstraction; arbitration logic is only generated when contention actually exists.
- 3) **Physical Limits:** At 8 cores, logic consumption surges ($O(N^2)$ behavior in the centralized driver's conflict matrix). This supports the need for distributed driver architectures in future work.

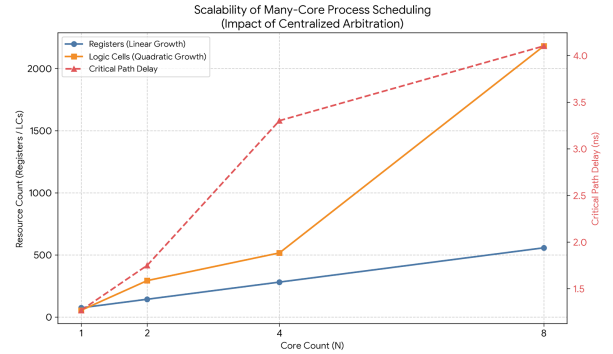


Fig. 6. Scalability Trends: Linear growth in state registers vs. quadratic growth in logic cells due to centralized arbitration.

E. Summary

- **Near-Zero Overhead:** For sequential control flow, HwOS matches handwritten RTL quality.
- **Controllable Cost:** Advanced concurrency adds minimal area (+4%) but requires attention to timing paths (26% delay).
- **System Benefits:** HwOS solves "state space explosion" in complex pipelines with extremely low code volume and high performance.

VII. DEBUGGING AND NATIVE OBSERVABILITY

In traditional hardware development workflows, debugging often implies tracing changes in thousands of signals in a Waveform Viewer. This "signal-level" debugging method loses the designer's high-level intent (e.g., FSM states, thread progress, resource holding status), resulting in low debugging efficiency. HwOS utilizes the global "God's Eye View" of its Kernel to build a native observability architecture from RTL to source code semantics, and implements a software-like interactive debugging experience via the accompanying HwOSgdb debugger.

A. Kernel: The Aggregation Center for Semantic Information

The HwOS Kernel is not only a resource manager but also an aggregator of debug information. During Chisel's Elaboration phase, the Kernel traverses the entire fractal architecture to establish mapping relationships between physical circuits and logical semantics.

- **Global ID Registry:** To flatten the tree-like Process-Thread structure for hardware monitoring, the Kernel maintains three core registries:
 - **TID (Thread ID):** Kernel assigns a unique TID to each instantiated `HardwareThread` via `registerThread` as an index for runtime monitoring.
 - **PID (Process ID):** Manages logic containers via `registerProcess` for reconstructing hierarchical views during debugging.
 - **DriverID:** Drivers mounted via the `mount` interface are assigned a `DriverID`. This allows the debugger to distinguish whether the current `Step` belongs to user thread logic or internal Driver micro-operations (e.g., `DriverStep`), thereby enabling visual tracing of system service calls.
- **Symbol Table Dump:** While generating Verilog, the Kernel calls `dumpSymbolTable` to generate a mapping file (`hwos.symbols`). This file records the mapping of (`ThreadName`, `PC`) $\rightarrow$ `StepName`, allowing the debugger to reverse-resolve obscure Program Counter (PC) values into human-readable labels defined in the source code, such as `Step("Fetch_Loop")`.

B. DPI Interface: Zero-Intrusion State Monitoring

To transmit hardware simulation states to the software debugger in real-time, HwOS constructs the

`KernelStateMonitor` using SystemVerilog's DPI-C mechanism.

Unlike traditional signal-level tracing, HwOSgdb provides source-level debugging similar to techniques proposed for HLS. This addresses the growing need for sophisticated bug localization tools in reconfigurable hardware. Furthermore, the high-level semantics of HwOS could potentially support formal verification, a field surveyed in [19] [20] [21]. The Kernel provides an `attachMonitor()` interface, which automatically scans all registered threads and aggregates their key state signals (`pc`, `active`, `start`, `abort`, `done`) onto a wide bus.

```
// Monitor probe generation logic in Kernel.scala
val pcVec    = VecInit(pc32Seq).asUInt
val activeVec = VecInit(threads.map(_.isRunning).
    toSeq).asUInt
// ...
val monitor = Module(new KernelStateMonitorDPI(
    nThreads))
```

Listing 4. Monitor probe generation logic in Kernel.scala

In simulation runtime, `KernelStateMonitorDPI` calls the C++ side function `kernel_monitor_tick` at every clock rising edge, pushing the complete System Snapshot of the current cycle to the software side. This mechanism achieves 100% observability of internal hardware states without modifying any user logic code.

C. HwOSgdb: Source-Level Hardware Debugger

Based on the symbol table provided by the Kernel and the DPI data stream, we developed HwOSgdb, a full-featured hardware debugger based on `ncurses`. It is not merely a replacement for a waveform viewer but an analysis tool that understands HwOS semantics.

- **Semantic Mapping and Visualization:** HwOSgdb reads `hwos.symbols` to display thread names and current `Step` names directly on the interface, rather than raw binary data. It can also syntax-highlight operations inside Drivers based on the `OwnerID` of the `Step` [22] (e.g., displaying ALU operations in cyan and memory operations in yellow), helping developers quickly identify whether an instruction is in a computation or memory access stage.
- **Time Travel:** Leveraging the flexibility of software simulation, HwOSgdb maintains a History Buffer. Developers can not only single-step execute (`Step`) but also "roll back" to previous cycles to view every frame of history that led to a deadlock or error, which is extremely effective for capturing transient errors. [23]
- **Expression Breakpoints:** The built-in expression parser supports semantic-based breakpoint settings, such as `break Thread_Fetch.pc == 2` or `break Core0/ALU.active == 1`. This allows developers to debug specific logic stages directly without manually calculating the timing of trigger conditions.

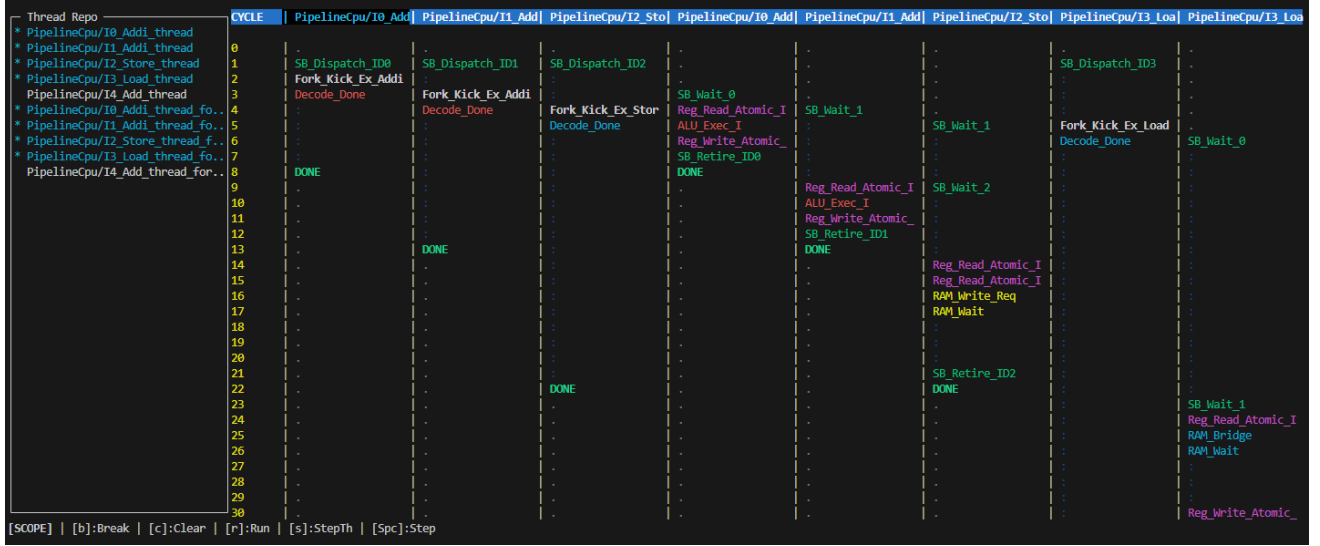


Fig. 7. HwOSgdb Interface: A terminal-based debugger providing source-level visibility into thread states, PC values, and active driver operations.

VIII. CONCLUSION AND FUTURE WORK

This paper proposes **HwOS**, a framework for constructing Thread-Level RTL (TL-RTL) oriented towards post-Moore era hardware design. Addressing the exponentially growing complexity of control logic in modern digital systems, HwOS abandons traditional low-level abstractions centered on signals and FSMs. Instead, it reconstructs the hardware description paradigm using a mature "Thread-Process-Driver" model derived from operating systems.

By introducing the **HardwareThread** primitive and the **ContextScope** metaprogramming mechanism, HwOS successfully decouples sequential logic from underlying circuit implementation, allowing designers to describe complex concurrent behaviors in an imperative mindset. The **Driver**-based control flow proxy model not only implements atomic resource access and implicit arbitration but also adapts upper-layer control flows to different underlying resource constraints without code modification via the "Instruction Injection" mechanism. The "Service-Based Pipeline" and "Fractal Many-Core Architecture" built upon this framework maintain area efficiency close to handwritten RTL while significantly reducing code volume and demonstrating excellent scalability. The accompanying HwOSgdb debugger further bridges the gap between RTL simulation and source-level semantics, filling the final piece of the agile hardware development puzzle.

The emergence of HwOS demonstrates that introducing high-level abstractions from software engineering into hardware design does not necessarily lead to performance or area losses. On the contrary, reasonable abstractions (such as TL-RTL) can release designers' innovative potential at the architectural level by eliminating the burden of artificial micromanagement.

Looking forward, we will deepen the research on HwOS in the following directions:

- 1) **Physical Design Awareness:** Exploring the introduction of physical layout information into the HwOS compiler backend to optimize timing convergence for long-distance control signals, supporting physical implementation from FPGA to ASIC.
- 2) **Distributed Driver Architecture:** Targeting ultra-large-scale many-core systems (> 64 cores), we will research distributed Driver implementations based on Network-on-Chip (NoC) to replace current centralized arbitration logic and alleviate routing congestion.
- 3) **Formal Verification Support:** Leveraging the high-level semantics of the HwOS thread model, we aim to explore the possibility of automatically generating formal verification properties, building a "Correct-by-Construction" trusted hardware generation flow.

We believe that as the wave of agile hardware development progresses, the "Everything is a Thread" philosophy advocated by HwOS will provide powerful tool support for next-generation architecture design.

SOURCE CODE AVAILABILITY

The source code of HwOS is available at: <https://github.com/NullStarfish/HwOS>.

REFERENCES

- [1] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [2] P. Flake, P. Moorby, S. Golson, A. Salz, and S. Davidmann, "Verilog hdl and its ancestors and descendants," *Proc. ACM Program. Lang.*, vol. 4, no. HOPL, Jun. 2020. [Online]. Available: <https://doi.org/10.1145/3386337>
- [3] L. Carloni, K. McMillan, and A. Sangiovanni-Vincentelli, "Theory of latency-insensitive design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 9, pp. 1059–1076, 2001.

- [4] P. Bjesse, K. Claessen, M. Sheeran, and S. Singh, "Lava: hardware design in haskell," in *Proceedings of the Third ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '98. New York, NY, USA: Association for Computing Machinery, 1998, p. 174–184. [Online]. Available: <https://doi.org/10.1145/289423.289440>
- [5] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, "Chisel: constructing hardware in a scala embedded language," in *Proceedings of the 49th Annual Design Automation Conference*, ser. DAC '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 1216–1225. [Online]. Available: <https://doi.org/10.1145/2228360.2228584>
- [6] R. Nikhil, "Bluespec system verilog: efficient, correct rtl from high level specifications," in *Proceedings. Second ACM and IEEE International Conference on Formal Methods and Models for Co-Design, 2004. MEMOCODE '04.*, 2004, pp. 69–70.
- [7] A. Izraelevitz, J. Koenig, P. Li, R. Lin, A. Wang, A. Magyar, D. Kim, C. Schmidt, C. Markley, J. Lawson, and J. Bachrach, "Reusability is firrtl ground: Hardware construction languages, compiler frameworks, and transformations," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, Nov 2017, pp. 209–216.
- [8] S. Jiang, P. Pan, Y. Ou, and C. Batten, "Pymtl3: A python framework for open-source hardware modeling, generation, simulation, and verification," *IEEE Micro*, vol. 40, no. 4, pp. 58–66, 2020.
- [9] A. Canis, J. Choi, M. Aldham, V. Zhang, A. Kammoona, J. H. Anderson, S. Brown, and T. Czajkowski, "Legup: high-level synthesis for fpga-based processor/accelerator systems," in *Proceedings of the 19th ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 33–36. [Online]. Available: <https://doi.org/10.1145/1950413.1950423>
- [10] R. Nane, V.-M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y. T. Chen, H. Hsiao, S. Brown, F. Ferrandi, J. Anderson, and K. Bertels, "A survey and evaluation of fpga high-level synthesis tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 10, pp. 1591–1604, 2016.
- [11] R. M. Tomasulo, "An efficient algorithm for exploiting multiple arithmetic units," *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25–33, 1967.
- [12] H. K.-H. So and R. Brodersen, "A unified hardware/software runtime environment for fpga-based reconfigurable computers using borph," *ACM Trans. Embed. Comput. Syst.*, vol. 7, no. 2, Jan. 2008. [Online]. Available: <https://doi.org/10.1145/1331331.1331338>
- [13] A. Agne, M. Happe, A. Keller, E. Lübbers, B. Plattner, M. Platzner, and C. Plessl, "Reconos: An operating system approach for reconfigurable computing," *IEEE Micro*, vol. 34, no. 1, pp. 60–71, 2014.
- [14] S. G. Tucker, "Microprogram control for system/360," *IBM Systems Journal*, vol. 6, no. 4, pp. 222–241, 1967.
- [15] Z. Liu, I. Calciu, M. Herlihy, and O. Mutlu, "Concurrent data structures for near-memory computing," in *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 235–245. [Online]. Available: <https://doi.org/10.1145/3087556.3087582>
- [16] B. W. Lampson, "Hints for computer system design," *SIGOPS Oper. Syst. Rev.*, vol. 17, no. 5, p. 33–48, Oct. 1983. [Online]. Available: <https://doi.org/10.1145/773379.806614>
- [17] C. A. R. Hoare, "Monitors: an operating system structuring concept," *Commun. ACM*, vol. 17, no. 10, p. 549–557, Oct. 1974. [Online]. Available: <https://doi.org/10.1145/355620.361161>
- [18] K. Yeager, "The mips r10000 superscalar microprocessor," *IEEE Micro*, vol. 16, no. 2, pp. 28–41, 1996.
- [19] N. Calagar, S. D. Brown, and J. H. Anderson, "Source-level debugging for fpga high-level synthesis," in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*, Sep. 2014, pp. 1–8.
- [20] J. Ma, G. Zuo, K. Loughlin, H. Zhang, A. Quinn, and B. Kasikci, "Debugging in the brave new world of reconfigurable hardware," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 946–962. [Online]. Available: <https://doi.org/10.1145/3503222.3507701>
- [21] C. Kern and M. R. Greenstreet, "Formal verification in hardware design: a survey," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 4, no. 2, p. 123–193, Apr. 1999. [Online]. Available: <https://doi.org/10.1145/307988.307989>
- [22] M. Weiser, "Program slicing," *IEEE Transactions on Software Engineering*, vol. SE-10, no. 4, pp. 352–357, 1984.
- [23] S. T. King, G. W. Dunlap, and P. M. Chen, "Debugging operating systems with time-traveling virtual machines," in *Proceedings of the 2005 USENIX Technical Conference*, 2005, pp. 1–15.